Javascript

Paulo Roberto Tazinazo Cândido

Desestruturação

```
const aluno = {matricula:123,nome:"João da Silva",dataNasc:"2000-02-23"}
console.log(aluno)

// Desestruturação
const {nome} = aluno
console.log(nome)

// Desestruturação
const {matricula:mt,dataNasc:dn} = aluno
console.log(dn)
```

- No primeiro exemplo de desestruturação é criada a constante “nome” contendo o nome do aluno.
- No segundo exemplo são criadas as constantes mt contendo a matricula e dn contendo a data de nascimento.

```
                                                    Exemplo.html:12
▼ {matricula: 123, nome: 'João da Silva', dataNasc: '2000-02-23'} i
    dataNasc: "2000-02-23"
    matricula: 123
    nome: "João da Silva"
  ▶ [[Prototype]]: Object
João da Silva                                       Exemplo.html:16
2000-02-23                                          Exemplo.html:20
```

Criar novo objeto com base em um existente

```
// Armazenando funções em constantes/variáveis
const f = function(a,b) {return a + b}
alert( f(10,20) )
```

Desestruturando um vetor

```
// Desestruturando um vetor
const vetor = [10,function(){return "ok"}]
const [x,y] = vetor
alert( y() )
```

- Criadas as constantes x com valor 10 e y contendo a função

Arrow Function

```
// Arrow function
const f1 = function(a,b) {return a + b}
const f2 = (a,b) => {return a + b}
const f3 = (a,b) => a + b  // para quando código é um simples return
// Exemplo de uso de template string
console.log( `Resultados: ${f1(1,2)}; ${f2(1,2)}; ${f3(1,2)};<br/>`)
```

Espalhamento

```
// Sem espalhamento
const e1 = {x:10,y:20}
const e2 = {e1,z:30}
console.log(e2)

// Com espalhamento
const e3 = {x:10,y:20}
const e4 = {...e1,z:30}
console.log(e4)
```

Resulta:
{e1:{x:10,y:20},z:30}

Resulta:
{x:10,y:20,z:30}

```
▼ {e1: {…}, z: 30} i
  ▶ e1: {x: 10, y: 20}
    z: 30
  ▶ [[Prototype]]: Object

▼ {x: 10, y: 20, z: 30} i
    x: 10
    y: 20
    z: 30
  ▶ [[Prototype]]: Object
```

Alterar json

```
// Alterando um valor
const k1 = {x:10,y:20}
k1.y=30
console.log(k1.y)

// Usando espalhamento
const k2 = {...k1,y:30}
console.log(k2.y)
```

Funções aninhadas

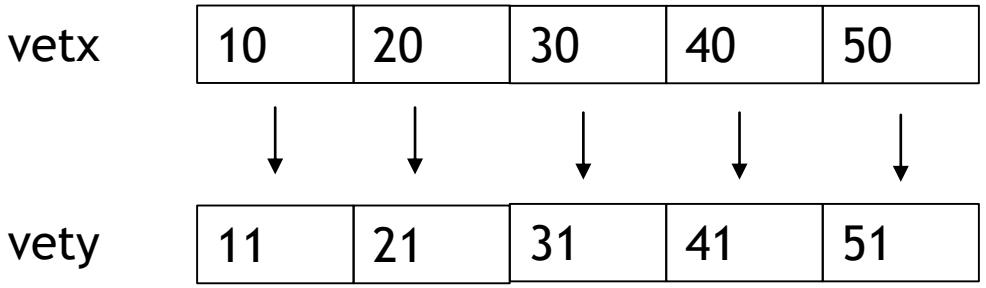
```
// Função aninhada
function fan1()
{
    function fan2(a,b=0)  // valor pdrão em b é zero
    {
        if(a>b) return a
        else return b
    }

    console.log( fan2(10,20) )
    console.log( fan2(30) )  // a será 30 e b será 0 (padrão)
}

// Teste
fan1()
```

Classes de Objetos

```
// Classes de objetos
class Aluno {
    constructor(m,n,d){
        this.matricula=m
        this.nome=n
        this.dataNasc=d
    }
    idade() // pode ser calculada com base na dataNasc, por isso o uso de uma função
    {
        const hoje = new Date()
        const dn = new Date(this.dataNasc)
        return (hoje - dn) / 1000 / 60 / 60 / 24 / 365
        // divisões acima para converter de milesegundos para ano
    }
}
// Teste
const aluno2 = new Aluno(456,'Maria Oliveira','2004-12-22')
console.log(aluno2.idade())
```

O objeto Aluno instanciado e atribuído a aluno2 é o JSON: {matricula:456,nome:”Maria Oliveira”,dataNasc:”2004-12-22”,idade: function(){ // código da função aqui }}

Copiar vetor transformando os dados




```
const vetx = [10,20,30,40,50]
const vety = vetx.map( (a)=>a+1  ) // Arrow function define a transformação
```

Cada valor de vetx é passado para o parâmetro “a” da arrow function e o resultado da arrow function corresponde ao valor em vety

Obs.: Pode ser usada função convencional em vez da arrow function.

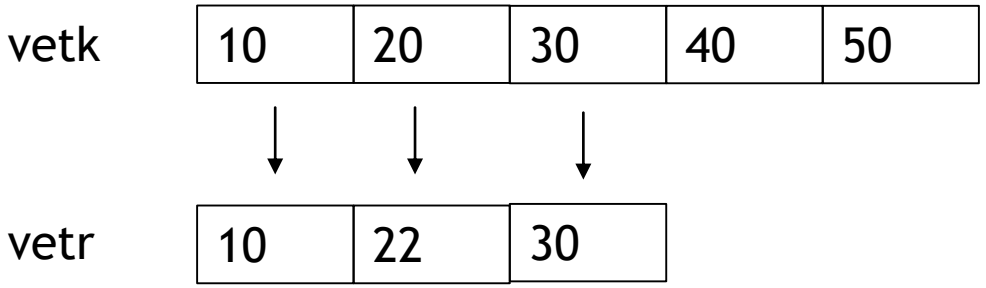
Totalizar os valores em um vetor

vetz	10	20	30	40	50

```
const vetz = [10,20,30,40,50]
const total = vetz.reduce( (acum,num)=>acum+num , 0)
console.log(total)
```

Enquanto o vetor é percorrido, cada valor é passado para o parâmetro “num” da arrow function. O parâmetro “acum” tem o valor das somas realizadas (soma acumulada) até a iteração atual. O segundo parâmetro de reduce, com valor “0”, é o valor inicial a ser somado junto com os valores do vetor.

Copiar vetor transformando os dados




```
const vetk = [10,20,30,40,50]
const vetr = vetk.filter( (x)=>x<=30 )
console.log(vetr)
```

A arrow function define os valores a serem mantidos no vetor. No caso: valores menores ou iguais a 30. Quando a arrow function returna true, o número e copiado para o novo vetor.